

# Department of Computer Science and Engineering

1. Subject Code: **SWE-401** Course Title: **Fundamentals of Software Engineering I**

2. **Expected Effort:** Total: 150 Hours Pacing: 10 Hours / Week Duration: 15 Weeks

3. **Prerequisites**

- Basic Computer Literacy, High School Algebra, Logical Problem Solving.

4. **Course Structure**

5 Units  $\times$  30 Hours (lectures, source-code reading, laboratory, PoW)

## 1. Course Description

This course takes a student with no web experience to a working full-stack engineer capable of shipping a secured, tested and containerised backend service with a React client. It deliberately spans both sides of the HTTP boundary so that the client-server contract is understood before distributed concerns are introduced in Software Engineering. Java and Spring Boot are the reference backend stack; PostgreSQL is the reference datastore; Docker Compose is the reference local environment. Every unit terminates in a running artefact that is carried forward, so by the end of the semester students hold a single continuously-evolved production-shaped application rather than five disconnected exercises.

## 2. Course Outcomes

After successful completion of the course, students will be able to:

- CO1.** Build responsive client-side user interfaces using modern HTML, CSS, JavaScript, and React.
- CO2.** Design and implement scalable RESTful APIs and modern communication protocols.
- CO3.** Apply enterprise-grade production practices including containerisation, logging, and security.
- CO4.** Utilise high-performance Java runtime capabilities and clean database interaction models.
- CO5.** Structure maintainable modular backend systems using clean architecture principles.

### 3. Detailed Syllabus

| Unit | Contents                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Hrs |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 1    | <p><b>Frontend Foundations &amp; the Client–Server Interface</b></p> <p><i>Module A: Web and HTTP Fundamentals</i><br/>How the web works: DNS resolution, TCP/TLS handshake, the HTTP request–response cycle, methods, headers, status codes, caching headers, and the browser rendering pipeline.</p> <p><i>Module B: Semantic Markup &amp; Layout Systems</i><br/>Semantic HTML5 and document structure. Accessibility fundamentals. The CSS box model, specificity, Flexbox and Grid layout systems, responsive breakpoints and mobile-first design.</p> <p><i>Module C: Modern JavaScript &amp; React</i><br/>ES6+ language features, the event loop, Promises and Async/Await, the Fetch API, modules and bundlers. Component-driven UI with React: state, props, hooks, lists and keys, controlled forms, and effect management.</p> <p><b>Hands-on Laboratory &amp; Proof-of-Work (PoW)</b></p> <ul style="list-style-type: none"><li>• <b>PoW:</b> Build a responsive interactive single-page dashboard in React that consumes a public API, handles loading and error states, and performs client-side form validation.</li><li>• Rebuild a supplied static design using Flexbox and Grid without any CSS framework.</li><li>• Instrument the browser Network panel to measure and explain request waterfalls.</li></ul> | 30  |
| 2    | <p><b>API Design, Relational Databases &amp; Core Backend</b></p> <p><i>Module A: RESTful API Design</i><br/>Resource modelling and URI design, correct status-code semantics, JSON serialisation, request validation, consistent error contracts, pagination and filtering, API versioning, and OpenAPI documentation.</p> <p><i>Module B: Relational Database Foundations</i><br/>PostgreSQL fundamentals. Schema design and normalisation, keys and constraints, joins, aggregation, transactions and isolation levels, indexing strategy, and reading query plans with <code>EXPLAIN ANALYZE</code>.</p> <p><i>Module C: Persistence in Java</i><br/>Spring Boot application structure and dependency injection. Object-Relational Mapping with Hibernate and Spring Data JPA: entity mapping, relationships, lazy loading pitfalls, repositories, and schema migrations.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 30  |

| Unit | Contents (continued)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Hrs |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
|      | <p><b>Hands-on Laboratory &amp; Proof-of-Work (PoW)</b></p> <ul style="list-style-type: none"> <li>• <b>PoW:</b> Design a multi-table relational schema for a task-management application and build a fully documented CRUD REST API using Java/Spring Boot.</li> <li>• Add indexes to a slow query and evidence the improvement with query plans and timings.</li> <li>• Configure environment-specific application profiles and externalised configuration.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |     |
| 3    | <p><b>Authentication, Authorisation &amp; Security Best Practices</b></p> <p><i>Module A: Credentials and Sessions</i><br/> Password storage with bcrypt/argon2, salting and work factors. Session-based versus token-based authentication. Cookie security flags, refresh strategies, and logout semantics.</p> <p><i>Module B: JWT Lifecycle &amp; Access Control</i><br/> JSON Web Token structure, signing and verification, expiry and rotation. Token verification filters in the request pipeline. Role-Based Access Control and fine-grained endpoint authorisation.</p> <p><i>Module C: Application Security Practice</i><br/> The OWASP Top 10 in practice: SQL Injection, XSS, CSRF, insecure direct object references, and mass assignment. CORS configuration, secret management, and basic rate limiting.</p> <p><b>Hands-on Laboratory &amp; Proof-of-Work (PoW)</b></p> <ul style="list-style-type: none"> <li>• <b>PoW:</b> Secure the task-management API with a robust registration and login flow, token verification filters, and fine-grained endpoint role authorisation.</li> <li>• Exploit and then patch a deliberately vulnerable build (injection, XSS, broken access control).</li> <li>• Configure CORS correctly for a browser client on a separate origin.</li> </ul> | 30  |
| 4    | <p><b>Production Readiness, Containerisation &amp; DevOps</b></p> <p><i>Module A: Software Testing Fundamentals</i><br/> Unit versus integration testing with JUnit 5. Test doubles and mocking, test data builders, database integration tests with Testcontainers, and coverage as a signal rather than a target.</p> <p><i>Module B: Build &amp; Configuration Management</i><br/> Maven/Gradle build lifecycles, dependency management, reproducible builds. Environment variable management and twelve-factor configuration discipline.</p> <p><i>Module C: Containerisation &amp; Local Orchestration</i><br/> Docker images and layers, writing efficient multi-stage Dockerfiles, image size and cache strategy. Docker Compose for local orchestration of application and database, healthchecks and startup ordering. Introduction to CI pipelines.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                     | 30  |

| Unit                              | Contents (continued)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Hrs        |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
|                                   | <b>Hands-on Laboratory &amp; Proof-of-Work (PoW)</b> <ul style="list-style-type: none"> <li>• <b>PoW:</b> Write automated integration tests for backend services, containerise both database and Java backend using Docker Compose, and verify seamless multi-container startup.</li> <li>• Reduce a naive container image size using multi-stage builds and measure the delta.</li> <li>• Wire a CI job that runs the test suite on every push.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |            |
| 5                                 | <b>Advanced Concurrency, Asynchronous Flows &amp; Production Patterns</b> <p><i>Module A: Java 21+ Concurrency</i><br/>Platform threads versus Virtual Threads. Simplifying asynchronous I/O execution, thread-per-request at scale, structured concurrency, and the pitfalls of pinning and thread-local state.</p> <p><i>Module B: Event-Driven Basics</i><br/>Message queues and publish-subscribe concepts, producer/consumer decoupling, delivery semantics, idempotent consumers, background jobs and scheduled tasks.</p> <p><i>Module C: Operability Patterns</i><br/>Centralised exception handling and error taxonomies. Structured JSON logging, correlation IDs, log levels and sampling. Health, readiness and metrics endpoints.</p> <b>Hands-on Laboratory &amp; Proof-of-Work (PoW)</b> <ul style="list-style-type: none"> <li>• <b>PoW:</b> Refactor blocking endpoints to leverage Java Virtual Threads and quantify throughput change under load.</li> <li>• <b>PoW:</b> Implement structured logging with correlation IDs and a resilient exception-handling framework for production tracking.</li> <li>• Publish and consume a domain event through a local broker with an idempotent consumer.</li> </ul> | 30         |
| <b>Total Instructional Effort</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | <b>150</b> |

#### 4. Recommended Resources

- Mozilla Developer Network (MDN) Web Docs — HTML, CSS, JavaScript reference.
- Walls, *Spring in Action*, latest edition.
- Kleppmann, *Designing Data-Intensive Applications* — Chapters 1–3 as background reading.
- Official React, PostgreSQL and Docker documentation.
- OWASP Top 10 and OWASP Cheat Sheet Series.